
Windows Forms

Lâm Hoài Bảo, FSE, CICT

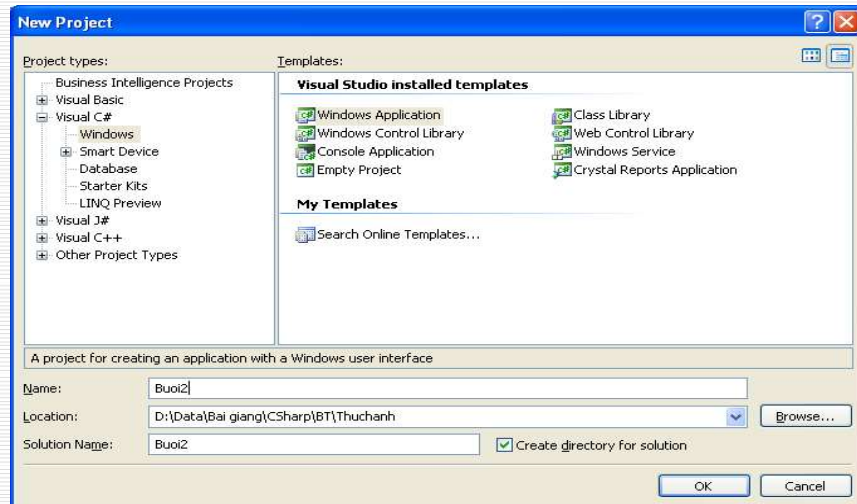
1

Agenda

- ☐ **Windows Forms application**
- ☐ Controls
- ☐ Validations
- ☐ MDI
- ☐ Summary

2

Project types

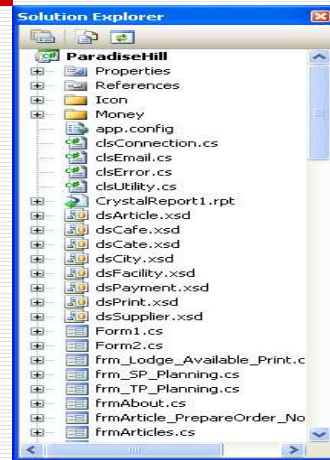


Windows Forms (WinForms)

- ☐ A collection of libraries to develop applications on Windows platform
- ☐ Namespace `System.Windows.Forms`
- ☐ Visual Studio
 - Choose New Project, Templates Windows Application

Visual Studio - Solution Explorer

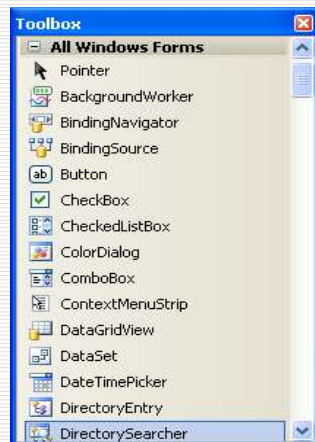
- ☐ Show current projects
- ☐ Represent hierarchical structure of files in each project



5

Visual Studio - ToolBox

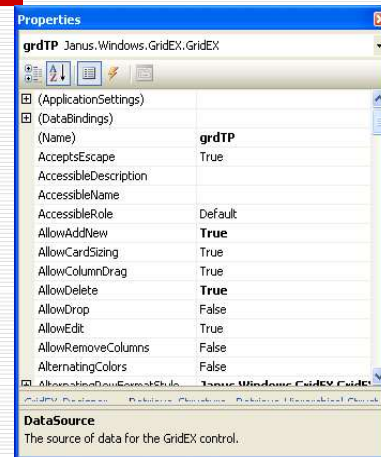
- ☐ Contain WinForms and other components to create application interfaces



6

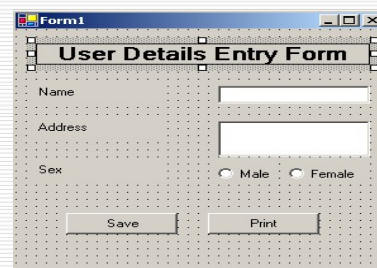
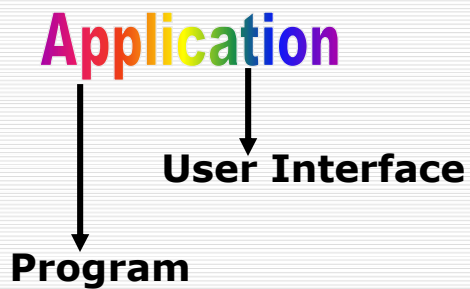
Visual Studio - Properties

- View and set property values of form, controls



7

WinForms application



```
private void button1_Click(object sender, System.EventArgs e)
{
    MessageBox.Show("Save");
}
```

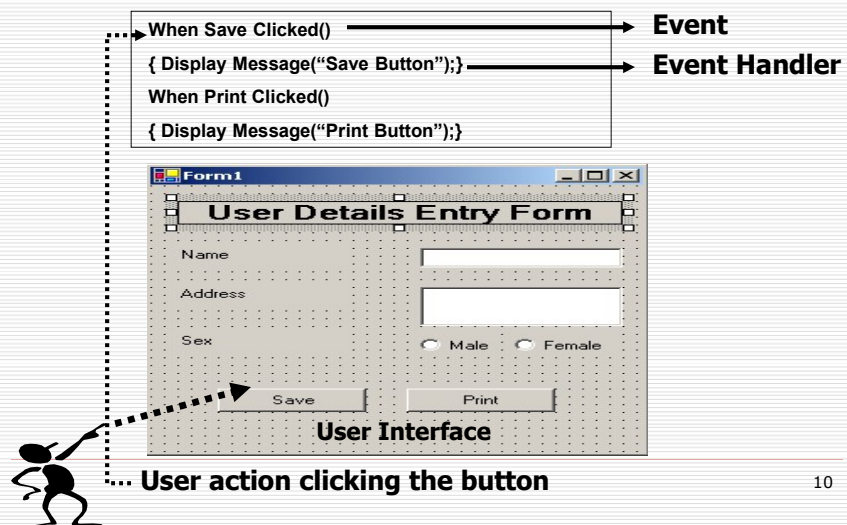
8

Components in a WinForms app

*.sln	Solution
*.cs	Code, form code generation
*.resx	Resource
*.csproj	Project
...	Others

9

Event and event handler



10

Agenda

- ☐ Windows Forms application
- ☐ **Controls**
- ☐ Validations
- ☐ MDI
- ☐ Summary

11

Form

12

Form

- ☐ Form: a programmed window to receive and display data
- ☐ A form is a class deriving from `System.Windows.Forms.Form`

13

Properties

- ☐ Common properties
 - `BackColor`: background color
 - `ForeColor`: font color
 - `Text`: form title
 - `Font`: Form font.
 - `Name`: form class name
- ☐ Values of those can be changed via code or properties window

14

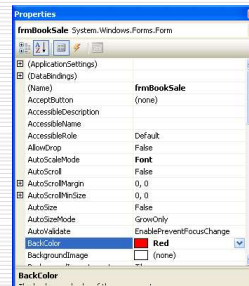
Form - Properties

□ Coding

<Form object>.<Property name> = <Value>;

■ Example: `this.BackColor = Color.Red;`

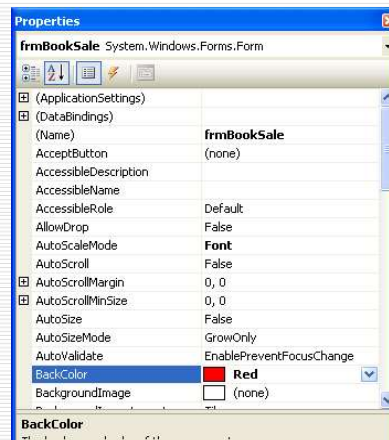
□ Property window



15

Form - Thuộc tính

□ Thay đổi thuộc tính của form bằng cửa sổ thuộc tính:



16

Form - Methods

- ☐ `Show()`: represent a form object
- ☐ `ShowDialog()`: represent a form object in dialog mode
- ☐ `Activate()` : activate a form object
- ☐ `Hide()`: hide a form object
- ☐ `Close()`: close a form object

17

Form - Methods

- ☐ Method call:
 <Form object>.<Method name> ([Parameter list])
- ☐ Example
 DialogForm myForm = **New** DialogForm();
 myForm.Show();
 myForm.ShowDialog();
 myForm.Activate();
 myForm.Hide();
 myForm.Close();

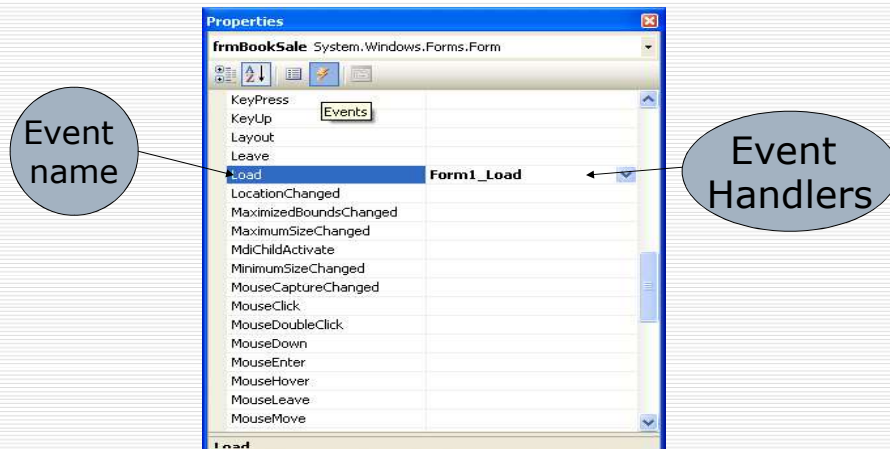
18

Form - Events

- Common events:
 - **Load**: Occur when the first instance of the form object is loaded
 - **Activated**: when the Form object is activated
 - **Deactivated**: when the form object is deactivated
 - **FormClosing**: when the form object is closing
- Usages
 - **Load**: initialize component values
 - **FormClosing** & **FormClosed**: release resources in a form object

19

Form event handlers



20

Form event handler

```
private void Form1_Load(object sender, EventArgs e)
{
    MessageBox.Show("Form Load");
}
```

21

Form event handler

- ☐ Prevent closing a form object:
 - FormClosing event handler

```
private void frmBookSale_FormClosing(object sender, FormClosingEventArgs e)
{
    e.Cancel=true;
}
```

22

Links form objects

- ☐ Create a form object
- ☐ Represent the form using Show()

```
<Form class> <object name> = new <Form clas>();  
<object name>.Show();  
  
ImageForm NewForm = new ImageForm();  
NewForm.Show();
```

23

Agenda

- ☐ Windows Forms application
- ☐ **Controls**
- ☐ Validations
- ☐ MDI

24

Controls

- ❑ Controls: components to create UI
- ❑ In WinForms, the base class is Control
- ❑ Common properties

Property	Description
BackColor	Background color
ForeColor	Font color
Name	Control object name
Enabled	Enable a form object or not
Visible	Visible a form object or not
Text	Text representing on the control

25

Access control members

- ❑ <Control object>.<Member name>
- ❑ Example:

```
button1.Enabled = true;
textBox1.Clear();
```

26

Controls on a form

- When controls are put on a form, they become members of the form
 - Field names: control names

27

Travel Entry

Around the World Travels

Traveler Name

Traveler Address

Desired Travel Type

Travel Duration

Desired Mode of Travel

Save

Clear

Exit

Label

TextBox

ComboBox

Listbox

Button

CheckedListBox

28

Controls on a form

```
partial class frmTravelEntry
{
    /// <summary>
    /// Required designer variable.
    /// </summary>
    private System.ComponentModel.IContainer components = null;

    /// <summary>
    /// Clean up any resources being used.
    /// </summary>
    /// <param name="disposing">true if managed resources should be disposed; otherwise, false;
    protected override void Dispose(bool disposing)
    {
        if (disposing)
        {
            // TODO: Add cleanup code here
        }
    }

    #region Windows Form Designer generated code
    private System.Windows.Forms.Label label1;
    private System.Windows.Forms.Label label2;
    private System.Windows.Forms.TextBox txtName;
    private System.Windows.Forms.TextBox txtAdd;
    private System.Windows.Forms.Label label3;
    private System.Windows.Forms.Label label4;
    private System.Windows.Forms.ComboBox cboTravelType;
    private System.Windows.Forms.Label label5;
    private System.Windows.Forms.ListBox lstDuration;
    private System.Windows.Forms.Label label6;
    private System.Windows.Forms.CheckedListBox lstMode;
    private System.Windows.Forms.Button btnSave;
    private System.Windows.Forms.Button btnClear;
    private System.Windows.Forms.Button btnExit;
    #endregion
}
```

Hungarian control naming (*)

- ☐ The name of a variable or a function indicates its intention (e.g type)
- ☐ A variable starts with a group of letters which are mnemonics for the type of the variable
 - Followed by a chosen name, normally in Proper Case
- ☐ Example

```
TextBox txtName;
Label lblInfor;
```

Hungarian control naming (*)

Class name	Prefix	Class name	Prefix
Lable	lbl	ListBox	lst (lsb)
Textbox	txt	ComboBox	cbo
Button	btn	GroupBox	grp
Checkbox	chk	PictureBox	pic
RadioButton	rad	ImageList	iml
TrackBar	trk	ListView	lvw
HScrollBar	hsb	VScrollBar	vsb
TreeView	trv	Form	frm
DatagridView	dgr	MenuStrip	msh

31

Label



☐ Display text

☐ Example

```
lblInformation.Text = "Some texts";
```

32

TextBox



- Receive/display text
- Properties
 - Multiline: multiline text or not (Default **false**).
 - CharacterCasing: automatically convert the input to the appropriate formats (Normal, Upper, Lower)
 - PasswordChar: masked character
 - Font: font of the textbox

33

TextBox



- **Methods**
 - Clear(): clear a textbox
 - Focus(): set the input focus to the object
- **Events**
 - KeyPress: Occurs when a character, space or backspace key is pressed while the control has focus
 - KeyDown: occurs when a key is pressed
 - KeyUp: occurs when a key is released

34

TextBox



□ Events

- Leave: occurs when the input focus leave the control
- Validating: occurs before the input focus leave the controls and validates the input

```
private void txtName_Validating(object sender, CancelEventArgs e)
{
    if(txtName.Text=="")//Nếu ô nhập là rỗng
        e.Cancel=true; // thì không thể di chuyển khỏi ô nhập
}
```

35

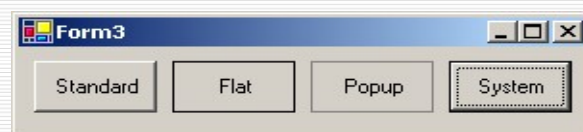
Button



- Confirm an action or operation

□ Properties

- Image: image on the button.
- FlatStyle: button style (Flat, Popup, Standard, and System.)



36

Button



- **Method**

- Focus(): sets the focus to the button

- **Event**

- Click: occurs when the button is clicked

37

ListBoxes

- Display a list

- 2 common listboxes:

- ListBox



- CheckedListBox



- A check before each element

38

ListBox



□ Properties

- Items: a collection of ListBoxItem.
- SelectedIndex: index of the current element (zero-based index)
- SelectedItems: a collection of chosen items.
- SelectedValue: value of the current element

□ Methods

- ClearSelected: clear of selections on a listbox.
- SetSelected(int, bool): select/unselect an item
- GetSelected(int): get the selected status of an item

39

ListBox



□ Events

- SelectedIndexChanged: occurs when the SelectedIndex property changes
- SelectedValueChanged: occurs when the SelectedValue property changes

□ Example:

- Use Items property to add to add elements to a listbox.
- Form_Load event handler



40

ListBox



```
private void frmTravelEntry_Load(object sender, EventArgs e)
{
    lstName.Items.Add("Nguyễn Văn Cường");
    lstName.Items.Add("Lâm Quang Hà");
    lstName.Items.Add("Trần Trung Trực");
    lstName.Items.Add("Lê Quang Trãi");
}
```

- ☐ Other members: Count, Clear, Insert, RemoveAt, Item...

41

CheckedListBox



- ☐ Some members are similar to those in ListBox
- ☐ **Properties**
 - CheckedExceptions: a collection of checked indices
 - CheckedItems: a collection of checked items

42

CheckedListBox



Methods

- SetItemChecked: establish a check to an item
- GetItemChecked: get a check status of an item
- SetItemCheckState: establish a checkstate of an item

Example



```
lstName.SetItemCheckState(1, CheckState.Checked);  
lstName.SetItemCheckState(2, CheckState.Indeterminate);  
lstName.SetItemCheckState(3, CheckState.Unchecked);
```

43

ComboBox



Combination of TextBox and ListBox

Properties

- DropDownStyle: ComboBox style: Simple, DropDown, DropDownList.



Simple



DropDown



DropDownList

44

- 45

46

```
// Hiển thị chi tiết
lsvName.View = View.Details;
//Sửa giá trị của Text
lsvName.LabelEdit = true;
lsvName.FullRowSelect = true;
// Hiển thị lưới
lsvName.GridLines = true;
ListViewItem item1 = new ListViewItem("Nguyễn Văn An", 0);
item1.SubItems.Add("1");
item1.SubItems.Add("2");
item1.SubItems.Add("3");

ListViewItem item2 = new ListViewItem("Lê Chí Công", 1);
item2.SubItems.Add("4");
item2.SubItems.Add("5");
item2.SubItems.Add("6");

ListViewItem item3 = new ListViewItem("Huỳnh Văn Hảo", 2);
item3.SubItems.Add("7");
item3.SubItems.Add("8");
item3.SubItems.Add("9");

// Tạo các cột cho các mục
lsvName.Columns.Add("Họ tên", -2, HorizontalAlignment.Left);
lsvName.Columns.Add("Toán", -2, HorizontalAlignment.Left);
lsvName.Columns.Add("Chuyên ngành", -2, HorizontalAlignment.Right);
lsvName.Columns.Add("Ngoại ngữ", -2, HorizontalAlignment.Center);
// Thêm các mục vào ListView
lsvName.Items.AddRange(new ListViewItem[] {item1, item2, item3});
```

TreeView



- ☐ Display hierarchical data
- ☐ TreeView support almost ListView members
- ☐ **Properties**
 - ImageList: a list of images of a TreeNode
 - Nodes: a list of TreeViewNodes.
 - SelectedNode: the current node

TreeView



□ Methods

- GetNodeAt: access a node at a position
- GetNodeCount: the number of nodes

□ Events

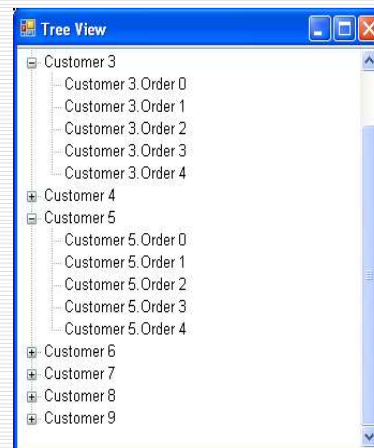
- BeforeSelect, AfterSelect: occurs before/after selecting a node
- BeforeCollapse, AfterCollapse: occurs before/after collapsing a node
- BeforeExpand, AfterExpand: occurs before/after expanding a node

49

TreeView



- Display 10 customers, each has an order
- Define Customer and Order classes
- Design a tree view



50

TreeView



```
class clsCustomer
{
    public string CustomerName;
    public ArrayList CustomerOrder;

    public clsCustomer(string Name)
    {
        CustomerName = Name;
        CustomerOrder = new ArrayList();
    }
}

class clsOrder
{
    public string OrderID;
    public clsOrder(string id)
    {
        OrderID=id;
    }
}
```

51

```
private void frmTreeView_Load(object sender, EventArgs e)
{
    ArrayList customerArray = new ArrayList();
    // Thêm khách hàng
    for(int x = 0; x<=9;x++)
        customerArray.Add(new clsCustomer("Customer " + x.ToString()));

    foreach(clsCustomer customer1 in customerArray)
        for(int y = 0; y<=4; y++)
            customer1.CustomerOrder.Add(new clsOrder("Order " + y.ToString()));

    // Xóa TreeView
    trvCusomer.Nodes.Clear();
    // Tạo các nút cha là các khách hàng

    foreach(clsCustomer customer1 in customerArray)
    {
        trvCusomer.Nodes.Add(new TreeNode(customer1.CustomerName));
        // Các nút con là các đơn đặt hàng

        foreach(clsOrder order1 in customer1.CustomerOrder)
            trvCusomer.Nodes[customerArray.IndexOf(customer1)].Nodes.Add
                (new TreeNode(customer1.CustomerName + "." + order1.OrderID));
    }
}
```

NumericUpDown



- A spin box displaying a numeric value
- **Properties**
 - Increment: the value to increment or decrement the spin box when the up or down buttons are clicked
 - Maximum, Minimum: the maximum (minimum) value of the NumericUpDown
 - Value: the current value of NumericUpDown

53

NumericUpDown



- **Methods**
 - DownButton, UpButton: decrease (increase) the current value of the spin box
- **Sự kiện**
 - ValueChanged: occurs when the spin box changes its value

54

CheckBox



- ☐ Receive/Display Yes/No information
- ☐ **Properties**
 - Checked: set/get the current status (true/false)
- ☐ **Events**
 - CheckChanged: occurs when the Checked property is changed its value

55

CheckBox



```
private void chkDam_CheckedChanged(object sender, EventArgs e)
{
    if (chkDam.Checked)
        txtName.Font = new Font("Arial", 10.0F, FontStyle.Bold);
    else
        txtName.Font = new Font("Arial", 10.0F, FontStyle.Regular);
}
```

56

RadioButton



- ☐ Receive/display Yes/No; but only select a single option of a group of choices
- ☐ Similar members as CheckBox.

57

GroupBox



- ☐ Group other controls
- ☐ **Example**

The screenshot shows a Windows form titled "FormDetails". At the top, there is a text box containing the text "You choose Core i3 running on Linux". Below the text box, there are two groups of radio buttons. The first group has three options: "Core i3" (selected), "Core i5", and "Core i7". The second group is titled "Operating system" and has three options: "Linux" (selected), "Mac OS", and "Windows".

58

PictureBox



- ☐ Display image
- ☐ **Property**
 - Image: image object
 - SizeMode: display mode (AutoSize, CenterImage, Normal, StretchImage)

59

ImageList



- ☐ Manage a collection of images
- ☐ **Properties**
 - Images: a collection of images
 - ImageSize: the size of the images

60

Timer



- ☐ Raise an event at each user-interval
- ☐ **Properties**
 - Enabled: get or set whether the timer is running
 - Interval: get or set the time duration (in ms) between 2 consecutive Tick events
- ☐ **Methods**
 - Start: start the timer
 - Stop: stop the timer
- ☐ **Event**
 - Tick: Occurs when the specified timer has elapsed and the timer is enabled

61

Timer



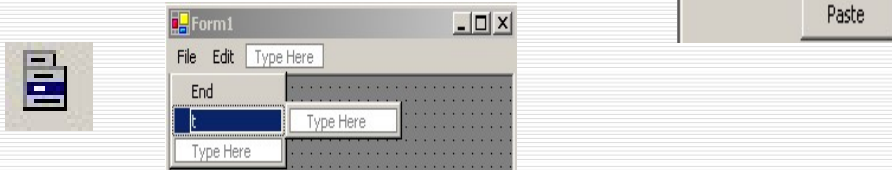
- ☐ **Thí dụ: Đồng hồ số:**
- ☐ **Properties:** Interval = 1000
Enabled = true
- ☐ **Tick event**

```
private void tmrDem_Tick(object sender, EventArgs e)
{
    lblTG.Text = DateTime.Now.ToString();
}
```

62

Menu

Types of Menu → ContextMenuStrip
↓
MenuStrip



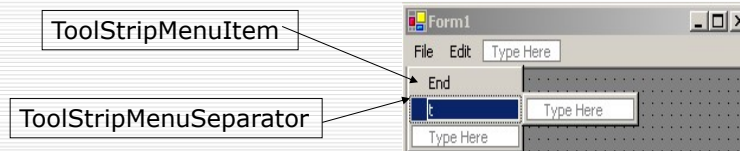
63

Menu

- ☐ Enable to hierarchically organize elements associated with command and event handlers
- ☐ Types
 - MenuStrip: provides a menu system of a form
 - ContextMenuStrip: popup menu that enable the control to expose functionality specified to a context

64

MenuStrip



65

ToolStripMenuItem

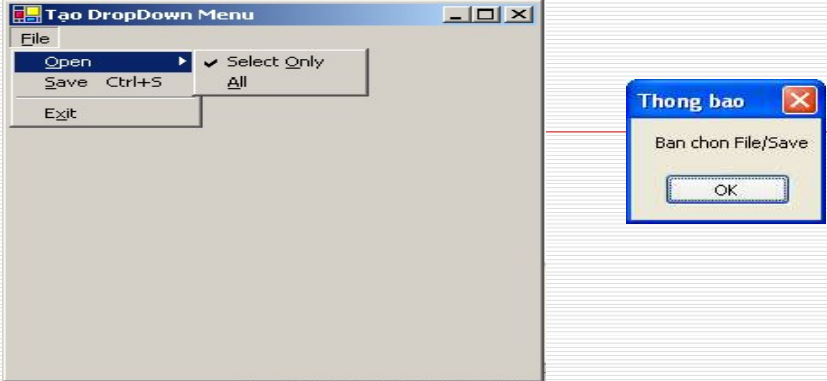
☐ Properties

- Checked: **True** if a check (✓) appears before a menu item
- Image: an image before a menu item
- ShortcutKeys: the shortcut key

☐ Event

- Click: When use click a menu item

66



```
private void mnuSave_Click(object sender, EventArgs e)
{
    MessageBox.Show("Ban chon File/Save", "Thông báo");
}

private void mnuExit_Click(object sender, EventArgs e)
{
    Application.Exit();
}
```

67

ContextMenuStrip

- ❑ Access menu items by a mouse right click
- ❑ After a ContextMenuStrip is created, it is attached to another control using the ContextMenuStrip property of that control

68

Other controls

- ☐ DateTimePicker
- ☐ LinkLabel
- ☐ TrackBar
- ☐ Panel
- ☐ TabControl
- ☐ RichTextBox
- ☐ ProgressBar
- ☐ ToolTip
- ☐ ToolBar
- ☐ StatusBar

69

Dialog

70

Dialog

Usage

- Display messages
- Receive messages

Type

Modal

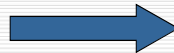
Modeless

71

Types

Custom dialog boxes

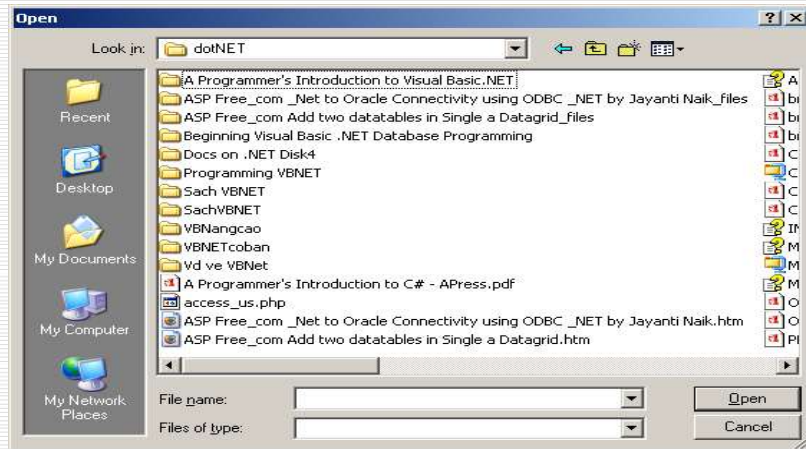
Common dialog boxes



- OpenFileDialog
- PageSetUpDialog
- FontDialog
- ColorDialog
- SaveFileDialog
- PrintPreviewDialog
- PrintDialog

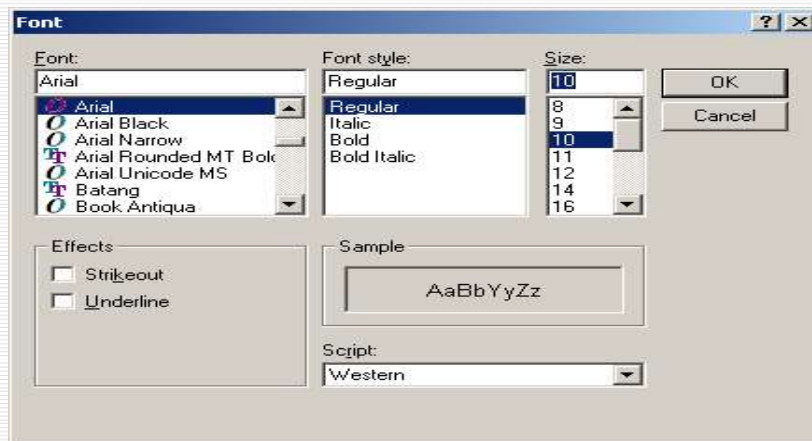
72

OpenFileDialog



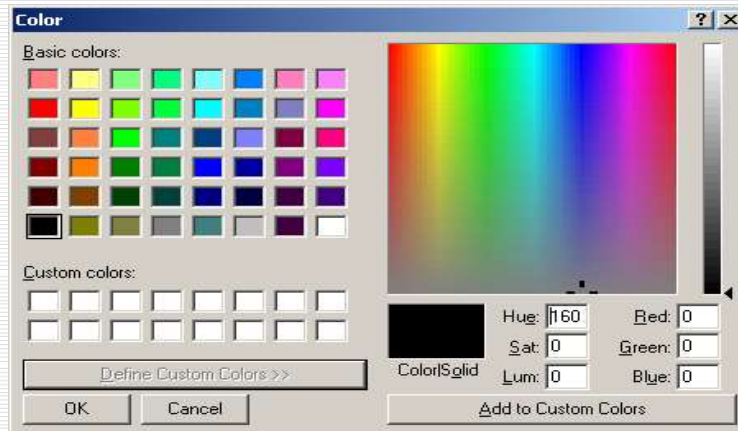
73

FontDialog



74

ColorDialog



75

MessageBox

- ☐ Display a message

```
MessageBox.Show("[Message]");
```

- ☐ The static Show() method of MessageBox class has 12 variants

76

MessageBox

Overload

<code>public static DialogResult Show(string);</code>
<code>public static DialogResult Show(IWin32Window, string);</code>
<code>public static DialogResult Show(string, string);</code>
<code>public static DialogResult Show(IWin32Window, string, string);</code>
<code>public static DialogResult Show(string, string, MessageBoxButtons);</code>
<code>public static DialogResult Show(IWin32Window, string, string, MessageBoxButtons);</code>
<code>public static DialogResult Show(string, string, MessageBoxButtons, MessageBoxIcon);</code>

77

MessageBox

Overload

<code>public static DialogResult Show(IWin32Window, string, string, string, MessageBoxButtons, MessageBoxIcon);</code>
<code>public static DialogResult Show(string, string, MessageBoxButtons, MessageBoxIcon, MessageBoxDefaultButton);</code>
<code>public static DialogResult Show(IWin32Window, string, string, MessageBoxButtons, MessageBoxIcon, MessageBoxDefaultButton);</code>
<code>public static DialogResult Show(string, string, MessageBoxButtons, MessageBoxIcon, MessageBoxDefaultButton, MessageBoxOptions);</code>
<code>public static DialogResult Show(IWin32Window, string, string, MessageBoxIcon, MessageBoxDefaultButton, MessageBoxOptions);</code>

78

MessageBox

- ☐ IWin32Window
- ☐ MessageBoxButtons
- ☐ MessageBoxIcon
- ☐ MessageBoxDefaultButton
- ☐ MessageBoxOptions

79

Agenda

- ☐ Windows Forms application
- ☐ Controls
- ☐ **Validations**
- ☐ MDI
- ☐ Summary

80

Data validation

Common events

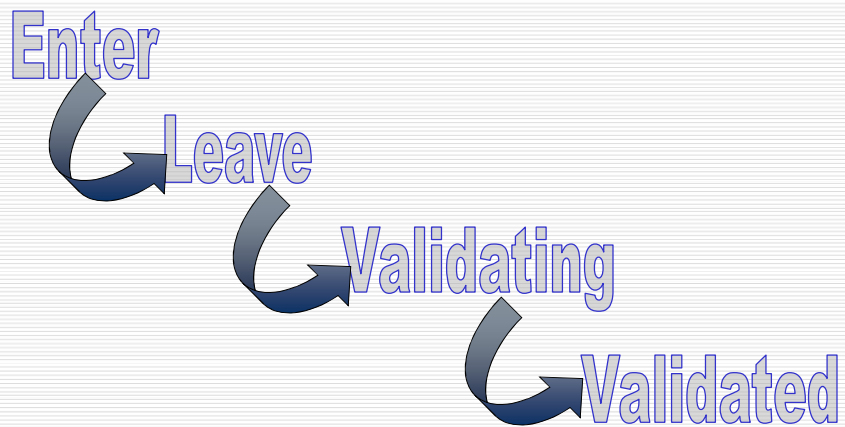
- Enter
- Leave
- Validating
- Validated

Validation type

- Form level
- Field level

81

Thứ tự các sự kiện



82

Field level

- ❑ Some events: Enter, Leave, Validating, Validated.
- ❑ Some keyboard events: KeyPress, KeyDown, KeyUp
- ❑ Some properties: MaxLength, Locked, PasswordChar.

83

Some Char methods

IsNumber
IsDigit
IsLetter
GetNumericValue
IsLetterOrDigit
IsLower
IsUpper
Parse

84

Form validation

- ☐ Validates all fields in a form
- ☐ Provides behaviors to invalid fields

85

ErrorProvider Control



- ☐ Display an error icon to indicate a control has an error
- ☐ **Properties**
 - BlinkRate: the rate which the error icon flashes.
 - BlinkStyle: error icon style (AlwaysBlink, BlinkDifferentError & NeverBlink).
 - Icon: the error icon
- ☐ **Methods**
 - GetError: get the error string on a specified control
 - SetError: set the error string to a control

86

ErrorProvider Control



```
private void btnTinh_Click(object sender, EventArgs e)
{
    try
    {
        int N = Convert.ToInt32(txtN.Text);
        long K=1;
        for(int i=1;i<=N;i++)
            K *= i;

        lblTG.Text=K.ToString();
    }
    catch(Exception ex)
    {
        errTB.SetError(txtN,ex.Message);
    }
}
```



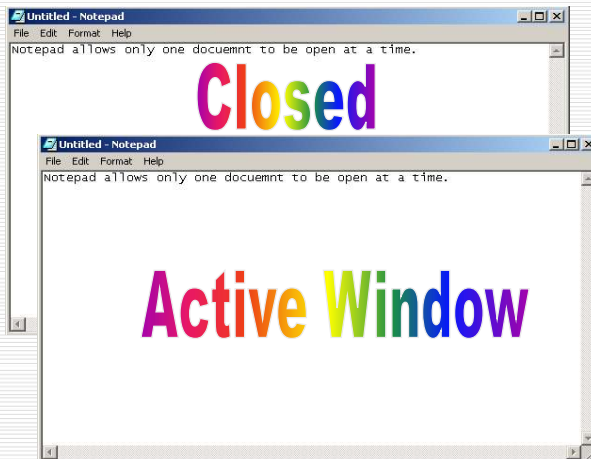
87

Agenda

- ☐ Windows Forms application
- ☐ Controls
- ☐ Validations
- ☐ **MDI**
- ☐ Summary

88

SDI - Single Document Interface



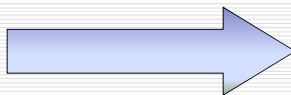
Allow only 1 window at a time

89

MDI - Multiple Document Interface



Documents

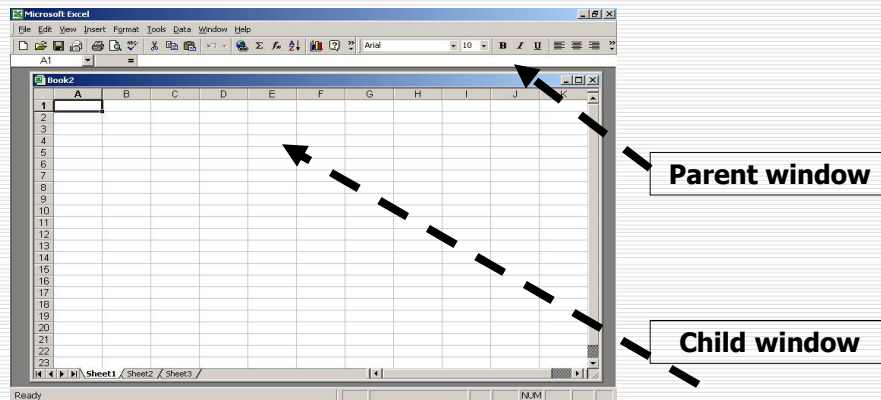


Group documents to a unit

- MDI allows multi-windows manipulation

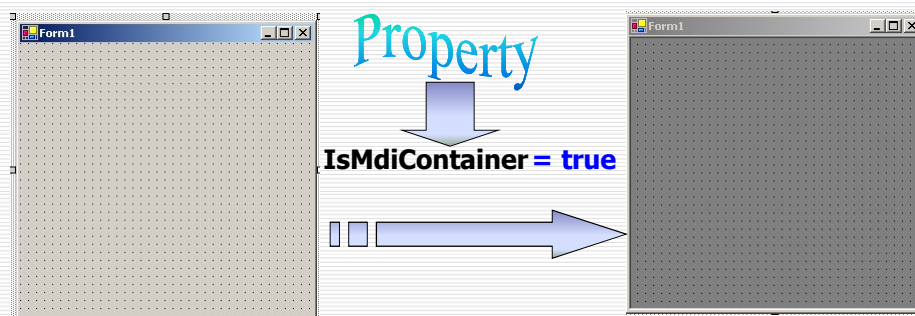
90

MDI



91

Create MDI form



92

Child form

- ❑ Use MdiParent:
 - <Child form object>.MdiParent = <Parent form object>;
 - <Child form object>.Show();
- ❑ **Example:**

```
frmTravelEntry f = new frmTravelEntry();  
f.MdiParent = this;  
f.Show();
```

93

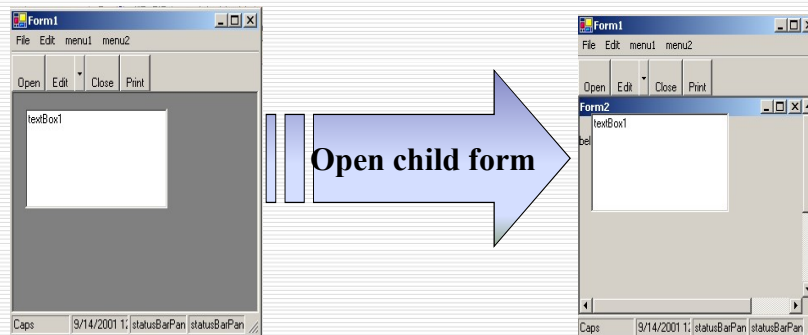
MDI characteristics

- ❑ Displays when the application starts
- ❑ Is a container
- ❑ Displays menu of child forms
- ❑ Only has 1 parent form in an application
- ❑ Opening many child forms is possible

94

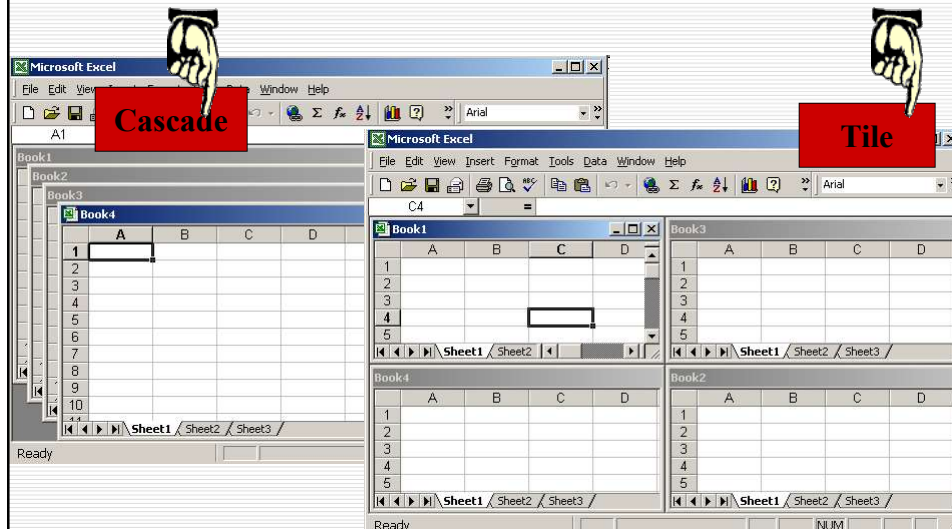
MDI characteristics

- ❑ Child forms can not move out the parent client rectangle
- ❑ The application is closed when closing the parent form



95

Child form sorting



Agenda

- ☐ Windows Forms application
- ☐ Controls
- ☐ Validations
- ☐ MDI
- ☐ **Summary**

97

Summary

- ☐ Windows Form template helps creating an application in Windows platform
- ☐ Event oriented programming
- ☐ Data validation can be done with the help of form design or coding

98